

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5113

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

I DINDU PUJITHA (192373037) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D PUJITHA

Annexure A
Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

Answer:

```
import hashlib
import hmac

key = b"AH_secret_key"

ip_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPSec"

mac = hmac.new(
    key,
    ip_packet,
    hashlib.sha256
).hexdigest()

print("IP Packet:")
print(ip_packet.decode())

print("\nAuthentication Code:")
print(mac)
```

```
received_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPSec"
```

```
received_mac = hmac.new(  
    key,  
    received_packet,  
    hashlib.sha256  
)hexdigest()
```

```
if hmac.compare_digest(mac, received_mac):  
    print("\nAH Verification: SUCCESS")  
    print("Packet integrity verified.")  
else:  
    print("\nAH Verification: FAILED")  
    print("Packet has been modified.")
```

```
modified_packet = b"192.168.1.10 -> 192.168.1.20 : Modified Data"
```

```
modified_mac = hmac.new(  
    key,  
    modified_packet,  
    hashlib.sha256  
)hexdigest()
```

```
if hmac.compare_digest(mac, modified_mac):  
    print("\nModified Packet: VALID")  
else:  
    print("\nModified Packet: INVALID")
```

```
print("Modification detected.")
```

Analysis

The program:

Creates an IP packet.

Generates an HMAC using a secret key.

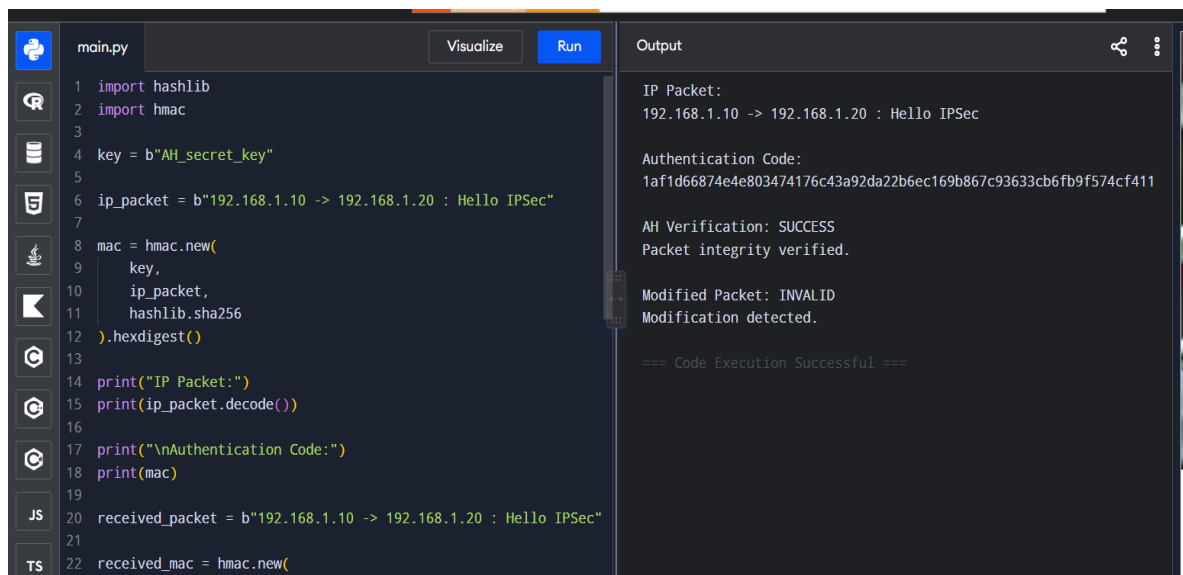
Sends the packet with its authentication code.

Recalculates the MAC at the receiver.

Compares both MAC values.

Detects packet modification.

Conclusion: AH provides **integrity and authentication**, but unlike ESP, it does not provide confidentiality.



```
main.py Visualize Run Output
1 import hashlib
2 import hmac
3
4 key = b"AH_secret_key"
5
6 ip_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPsec"
7
8 mac = hmac.new(
9     key,
10    ip_packet,
11    hashlib.sha256
12).hexdigest()
13
14 print("IP Packet:")
15 print(ip_packet.decode())
16
17 print("\nAuthentication Code:")
18 print(mac)
19
20 received_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPsec"
21
22 received_mac = hmac.new(
23     key,
24     received_packet,
25     hashlib.sha256
26).hexdigest()
27
28 if mac == received_mac:
29     print("AH Verification: SUCCESS")
30     print("Packet integrity verified.")
31 else:
32     print("Modified Packet: INVALID")
33     print("Modification detected.")
34
35 == Code Execution Successful ==
```

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

Answer:

```
key = 123

payload = "This is a confidential IP packet payload."

print("Original Payload:")
print(payload)

encrypted = []

for character in payload:
    encrypted.append(ord(character) ^ key)

print("\nEncrypted Payload:")
print(encrypted)

decrypted = ""

for value in encrypted:
    decrypted += chr(value ^ key)

print("\nDecrypted Payload:")
print(decrypted)
```

```
if decrypted == payload:
    print("\nESP Verification: SUCCESS")
    print("Payload decrypted successfully.")
else:
    print("\nESP Verification: FAILED")
```

Analysis

Original Data



Encryption



Encrypted Payload



Transmission

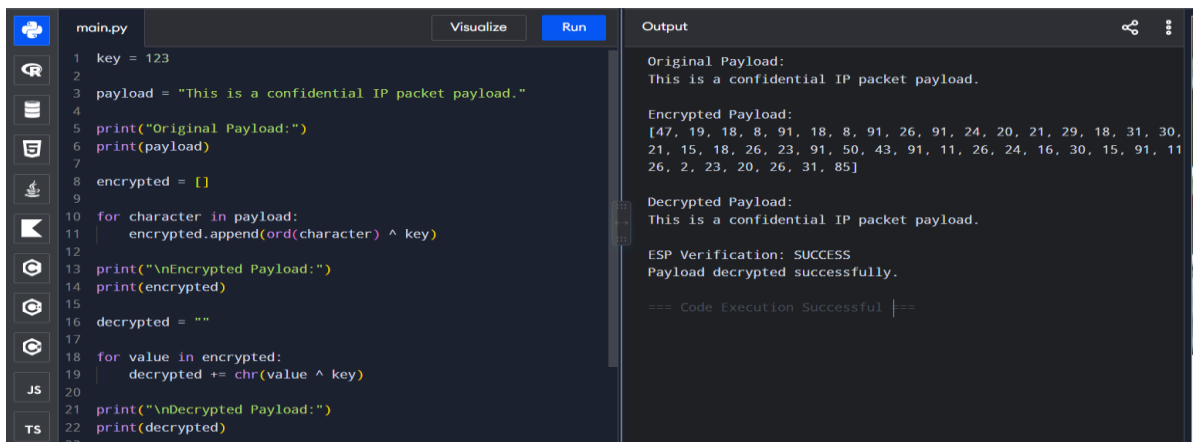


Decryption



Original Data

Conclusion: ESP protects the confidentiality of IP packet payloads. In actual IPSec, secure algorithms such as **AES**



```
main.py Visualize Run
1 key = 123
2
3 payload = "This is a confidential IP packet payload."
4
5 print("Original Payload:")
6 print(payload)
7
8 encrypted = []
9
10 for character in payload:
11     encrypted.append(ord(character) ^ key)
12
13 print("\nEncrypted Payload:")
14 print(encrypted)
15
16 decrypted = ""
17
18 for value in encrypted:
19     decrypted += chr(value ^ key)
20
21 print("\nDecrypted Payload:")
22 print(decrypted)
```

Output

```
Original Payload:
This is a confidential IP packet payload.

Encrypted Payload:
[47, 19, 18, 8, 91, 18, 8, 91, 26, 91, 24, 20, 21, 29, 18, 31, 30,
21, 15, 18, 26, 23, 91, 50, 43, 91, 11, 26, 24, 16, 30, 15, 91, 11
26, 2, 23, 20, 26, 31, 85]

Decrypted Payload:
This is a confidential IP packet payload.

ESP Verification: SUCCESS
Payload decrypted successfully.

=== Code Execution Successful ===
```

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

Answer:

```
security_associations = {}

def create_sa(spi, source, destination, algorithm):
    security_associations[spi] = {
        "source": source,
        "destination": destination,
        "algorithm": algorithm
    }

    print("Security Association created.")
    print("SPI:", spi)

def lookup_sa(spi):
    if spi in security_associations:
        print("\nSecurity Association Found:")
        print(security_associations[spi])
    else:
        print("\nSecurity Association not found.")

def delete_sa(spi):
    if spi in security_associations:
        del security_associations[spi]
```

```
        print("\nSecurity Association deleted.")
    else:
        print("\nSecurity Association not found.")

def display_sas():
    print("\nCurrent Security Associations:")

    if not security_associations:
        print("No Security Associations available.")
    else:
        for spi, details in security_associations.items():
            print("SPI:", spi)
            print("Source:", details["source"])
            print("Destination:", details["destination"])
            print("Algorithm:", details["algorithm"])
            print()

create_sa(
    1001,
    "192.168.1.10",
    "192.168.1.20",
    "AES-256"
)

create_sa(
    1002,
```



```

        "192.168.1.30",
        "192.168.1.40",
        "AES-256"
    )

    display_sas()

    lookup_sa(1001)

    delete_sa(1001)

display_sas()

```

Analysis

A Security Association stores security parameters such as:

- **SPI (Security Parameters Index)**
- Source address
- Destination address

The screenshot shows a code editor with a file named `main.py` and an output window. The code defines functions to create, lookup, and delete Security Associations (SAs). The output window shows the results of running the code, including the creation of an SA, a lookup for a specific SPI, and the deletion of an SA.

```

1 security_associations = {}
2
3 def create_sa(spi, source, destination, algorithm):
4     security_associations[spi] = {
5         "source": source,
6         "destination": destination,
7         "algorithm": algorithm
8     }
9
10    print("Security Association created.")
11    print("SPI:", spi)
12
13
14    def lookup_sa(spi):
15        if spi in security_associations:
16            print("\nSecurity Association Found:")
17            print(security_associations[spi])
18        else:
19            print("\nSecurity Association not found.")
20
21
22    def delete_sa(spi):

```

Output:

```

Algorithm: AES-256

SPI: 1002
Source: 192.168.1.30
Destination: 192.168.1.40
Algorithm: AES-256

Security Association Found:
{'source': '192.168.1.10', 'destination': '192.168.1.20',
'algorithm': 'AES-256'}

Security Association deleted.

Current Security Associations:
SPI: 1002
Source: 192.168.1.30
Destination: 192.168.1.40
Algorithm: AES-256

=== Code Execution Successful ===

```

4. **Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.**

Answer:

```
import hashlib

p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

message = "Hello, this is a secure email."

print("Original Email:")
print(message)

hash_value = int(
    hashlib.sha256(message.encode()).hexdigest(),
    16
) % n

signature = pow(hash_value, d, n)

print("\nDigital Signature:")
```

```
print(signature)

key = 23

encrypted = []

for character in message:
    encrypted.append(ord(character) ^ key)

print("\nEncrypted Email:")
print(encrypted)

decrypted = ""

for value in encrypted:
    decrypted += chr(value ^ key)

print("\nDecrypted Email:")
print(decrypted)

received_hash = int(
    hashlib.sha256(decrypted.encode()).hexdigest(),
    16
) % n

verification = pow(signature, e, n)

if verification == received_hash:
```

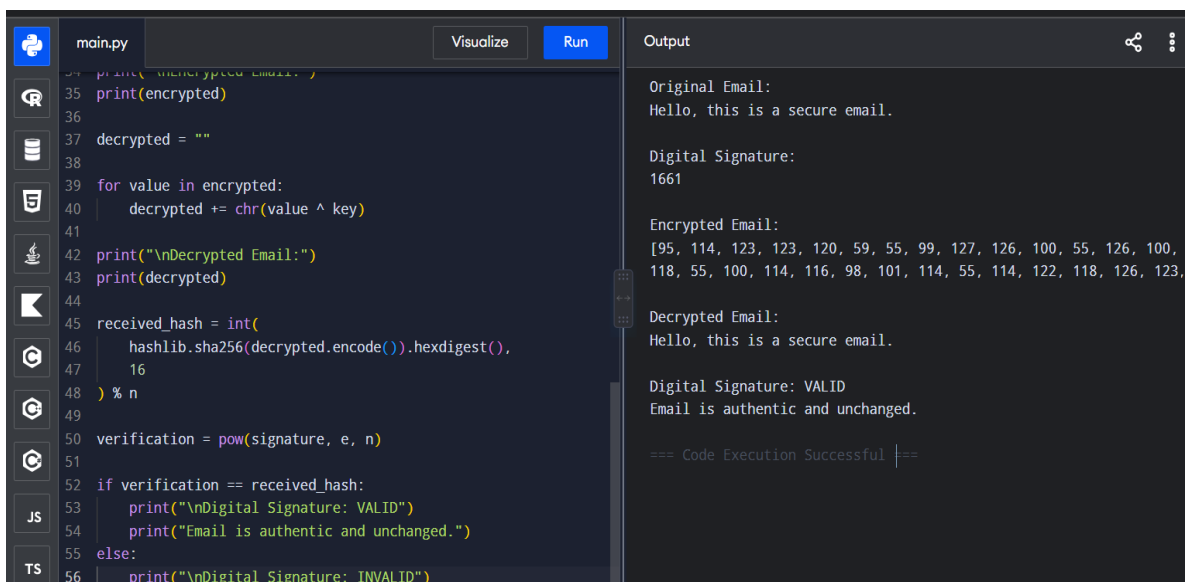
```
print("\nDigital Signature: VALID")
print("Email is authentic and unchanged.")
else:
print("\nDigital Signature: INVALID")
```

Analysis

A PGP system combines:

- **Encryption** → confidentiality
- **Hashing** → integrity
- **Digital signatures** → authentication

Conclusion: PGP provides secure email communication by combining encryption and digital signatures. This program is a simplified educational model rather than interoperable OpenPGP software.



The screenshot shows a code editor with a file named `main.py`. The code implements a simplified PGP system. It starts by printing the encrypted email. Then, it decrypts the email using a XOR operation with a key. Next, it calculates a SHA256 hash of the decrypted email and compares it with a received hash. If the hashes match, it prints a valid digital signature; otherwise, it prints an invalid one. The output window on the right shows the execution results: the original email, the encrypted email (a list of integers), the decrypted email, and the valid digital signature.

```
main.py Visualize Run
35 print(encrypted)
36
37 decrypted = ""
38
39 for value in encrypted:
40     decrypted += chr(value ^ key)
41
42 print("\nDecrypted Email:")
43 print(decrypted)
44
45 received_hash = int(
46     hashlib.sha256(decrypted.encode()).hexdigest(),
47     16
48 ) % n
49
50 verification = pow(signature, e, n)
51
52 if verification == received_hash:
53     print("\nDigital Signature: VALID")
54     print("Email is authentic and unchanged.")
55 else:
56     print("\nDigital Signature: INVALID")
```

Output

```
Original Email:
Hello, this is a secure email.

Digital Signature:
1661

Encrypted Email:
[95, 114, 123, 123, 120, 59, 55, 99, 127, 126, 100, 55, 126, 100,
118, 55, 100, 114, 116, 98, 101, 114, 55, 114, 122, 118, 126, 123,

Decrypted Email:
Hello, this is a secure email.

Digital Signature: VALID
Email is authentic and unchanged.

=== Code Execution Successful ===
```

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

Answer:

```
spam_words = [  
    "free",  
    "winner",  
    "prize",  
    "money",  
    "offer",  
    "urgent",  
    "click here",  
    "lottery",  
    "congratulations"  
]  
  
blacklisted_senders = [  
    "spam@example.com",  
    "winner@example.com"  
]  
  
def classify_email(sender, subject, body):  
    score = 0
```

```
if sender.lower() in blacklisted_senders:
    score += 3

text = (subject + " " + body).lower()

for word in spam_words:
    if word in text:
        score += 1

if score >= 3:
    return "SPAM"
else:
    return "LEGITIMATE"

sender = input("Enter sender email: ")
subject = input("Enter email subject: ")
body = input("Enter email body: ")

result = classify_email(sender, subject, body)

print("\nEmail Classification:", result)
```

Analysis

The program analyzes:

Email headers:

- Sender address
- Blacklisted sender

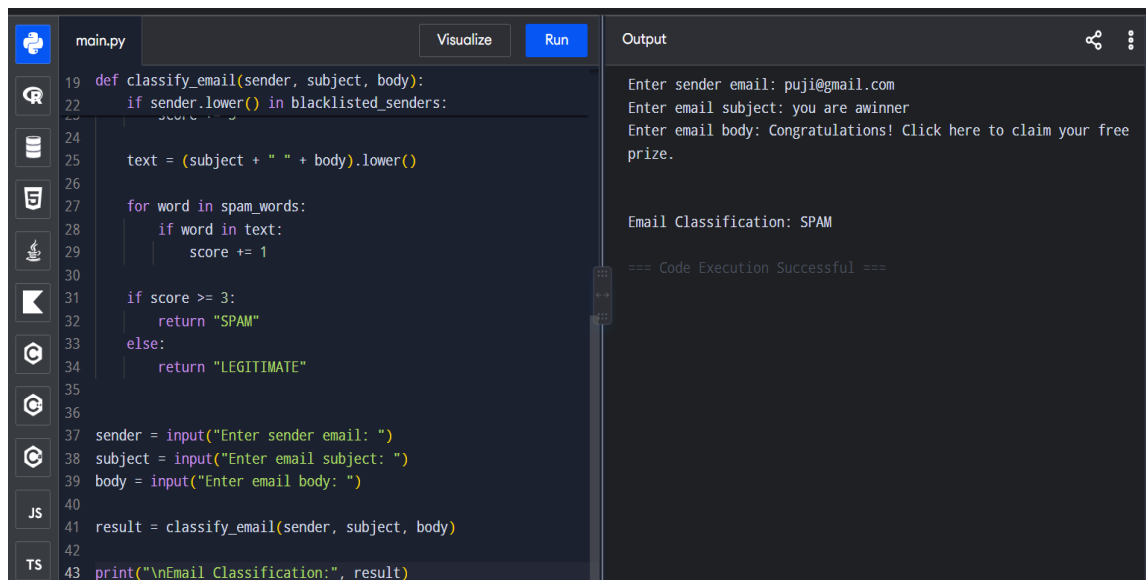
Email content:

- Spam-related keywords
- Suspicious phrases

A score is calculated based on detected indicators. If the score reaches a threshold, the email is classified as spam.

Conclusion

Rule-based spam filtering is simple and fast, but it can have difficulty detecting new spam patterns. More advanced s



The image shows a code editor interface with a file named 'main.py'. The code defines a function 'classify_email' that checks if the sender is in a blacklisted list and then iterates through a list of spam words to calculate a score. If the score is 3 or higher, it returns 'SPAM'; otherwise, it returns 'LEGITIMATE'. The script then prompts the user for sender email, subject, and body, and prints the classification result. The output window shows the user input: 'puji@mail.com', 'you are awinner', and 'Congratulations! Click here to claim your free prize.', followed by the output 'Email Classification: SPAM' and a success message '=== Code Execution Successful ==='.

```
main.py Visualize Run
19 def classify_email(sender, subject, body):
20     score = 0
21     if sender.lower() in blacklisted_senders:
22         score += 1
23
24     text = (subject + " " + body).lower()
25
26     for word in spam_words:
27         if word in text:
28             score += 1
29
30     if score >= 3:
31         return "SPAM"
32     else:
33         return "LEGITIMATE"
34
35
36
37 sender = input("Enter sender email: ")
38 subject = input("Enter email subject: ")
39 body = input("Enter email body: ")
40
41 result = classify_email(sender, subject, body)
42
43 print("\nEmail Classification:", result)
```

Output

```
Enter sender email: puji@mail.com
Enter email subject: you are awinner
Enter email body: Congratulations! Click here to claim your free
prize.

Email Classification: SPAM

=== Code Execution Successful ===
```